You said:

NEWACC_Architect...
Document

Bạn giúp mình diễn giải và đánh giá về giải pháp trên

Worked for 14s

ChatGPT said:

Mình đã đọc toàn bộ tài liệu **NEWACC_Architecture_v3.md, phiên bản 3.1 ngày 01/11/2026**. Đây là một tài liệu khá lớn, bao quát từ kiến trúc phần mềm, mô hình dữ liệu kế toán, quy trình ghi sổ, AI Agent cho đến khả năng mở rộng đa cơ sở dữ liệu và kết nối agent bên ngoài.

Mình muốn đánh giá nó ở ba góc độ: **bản chất giải pháp, giá trị kiến trúc và tính khả thi khi triển khai**. Đặc biệt, mình sẽ không chỉ đánh giá những gì tài liệu tuyên bố mà còn xem xét những gì có thể xảy ra khi đưa vào vận hành thực tế.

I. Diễn giải: NEWACC thực chất đang giải quyết bài toán gì?

Theo mình, đây không đơn thuần là việc tích hợp AI vào một phần mềm kế toán.

Đây là một nỗ lực chuyển NEWACC từ một ứng dụng kế toán truyền thống sang một **nền tảng nghiệp vụ kế toán có khả năng tiếp nhận, xử lý và cộng tác với nhiều loại tác nhân thông minh**.

Có thể diễn giải mục tiêu này bằng một sơ đồ đơn giản:

Mô hình NEWACC

Các nguồn dữ liệu

Hóa đơn điện tử · Excel · OCR · Người dùng · AI · Agent bên ngoài

↓

STAGING – Lớp đệm

VoucherDraft

Tiếp nhận · Chuẩn hóa · Mapping · Đối chiếu · Rà soát

↓

CONVERSION & VALIDATION

Sinh định khoản · Kiểm tra nghiệp vụ · Người dùng phê duyệt

↓

IPostingGateway

Cổng ghi sổ duy nhất

Transaction · Validation · Audit

↓

CORE KẾ TOÁN

JournalEntry · Inventory · Debt · VAT · General Ledger · Reports

Có bốn ý tưởng lớn nằm phía sau mô hình này.

1. Tách biệt trí tuệ và quyền quyết định nghiệp vụ

Đây là điểm quan trọng nhất.

Giải pháp không giao cho AI quyền quyết định số liệu kế toán. AI chỉ được phép:

- Đọc dữ liệu.
- Tìm kiếm đối tượng, vật tư, tài khoản.
- Phân tích lịch sử hạch toán.
- Đưa ra đề xuất mapping.
- Tạo bản nháp chứng từ.
- Giải thích các vấn đề phát hiện được.

Người dùng mới là người quyết định cuối cùng và thực hiện ghi sổ.

Điều này tạo ra một ranh giới khá rõ:

Thành phần	Vai trò
AI Agent	Phân tích, suy luận, đề xuất
Core kế toán	Tính toán, kiểm tra, quyết định tính hợp lệ

Người dùng	Xác nhận và chịu trách nhiệm ghi sổ

Mình đánh giá đây là một lựa chọn đúng về mặt kiến trúc lẫn quản trị rủi ro.

Một mô hình AI có thể rất giỏi đọc hóa đơn nhưng không vì thế mà được phép tự quyết định một bút toán 500 triệu đồng.

2. Staging – một lớp trung gian có ý nghĩa lớn hơn việc lưu nháp

Theo mình, đây là phần có giá trị kiến trúc nổi bật nhất.

Tài liệu xây dựng `VoucherDraft` thành một định dạng trung gian thống nhất cho tất cả các nguồn dữ liệu.

Đặc biệt, draft có ba vòng đời:

Giai đoạn	Nội dung	Ý nghĩa
Stage 1	Raw	Dữ liệu nguyên thủy
Stage 2	Resolution	Dữ liệu đã chuẩn hóa và mapping
Stage 3	Projection	Dữ liệu đã được chuyển thành cấu trúc hạch toán

Điểm đáng chú ý là ba thành phần `raw`, `resolved`, `match` được lưu riêng.

Ví dụ:

Một hóa đơn ghi tên hàng là "Giấy A4 Double A".

- `raw` : Giấy A4 Double A.
- `resolved` : VPP-A4.
- `match` : Được xác định thông qua bí danh đã học, độ tin cậy 0,98.

Điều này cho phép hệ thống biết được không chỉ kết quả mà cả nguồn gốc của kết quả.

Nếu sau này kế toán phát hiện mapping sai, có thể truy lại vì sao hệ thống đưa ra đề xuất đó.

Đây là một đặc điểm rất có giá trị đối với phần mềm kế toán có AI.

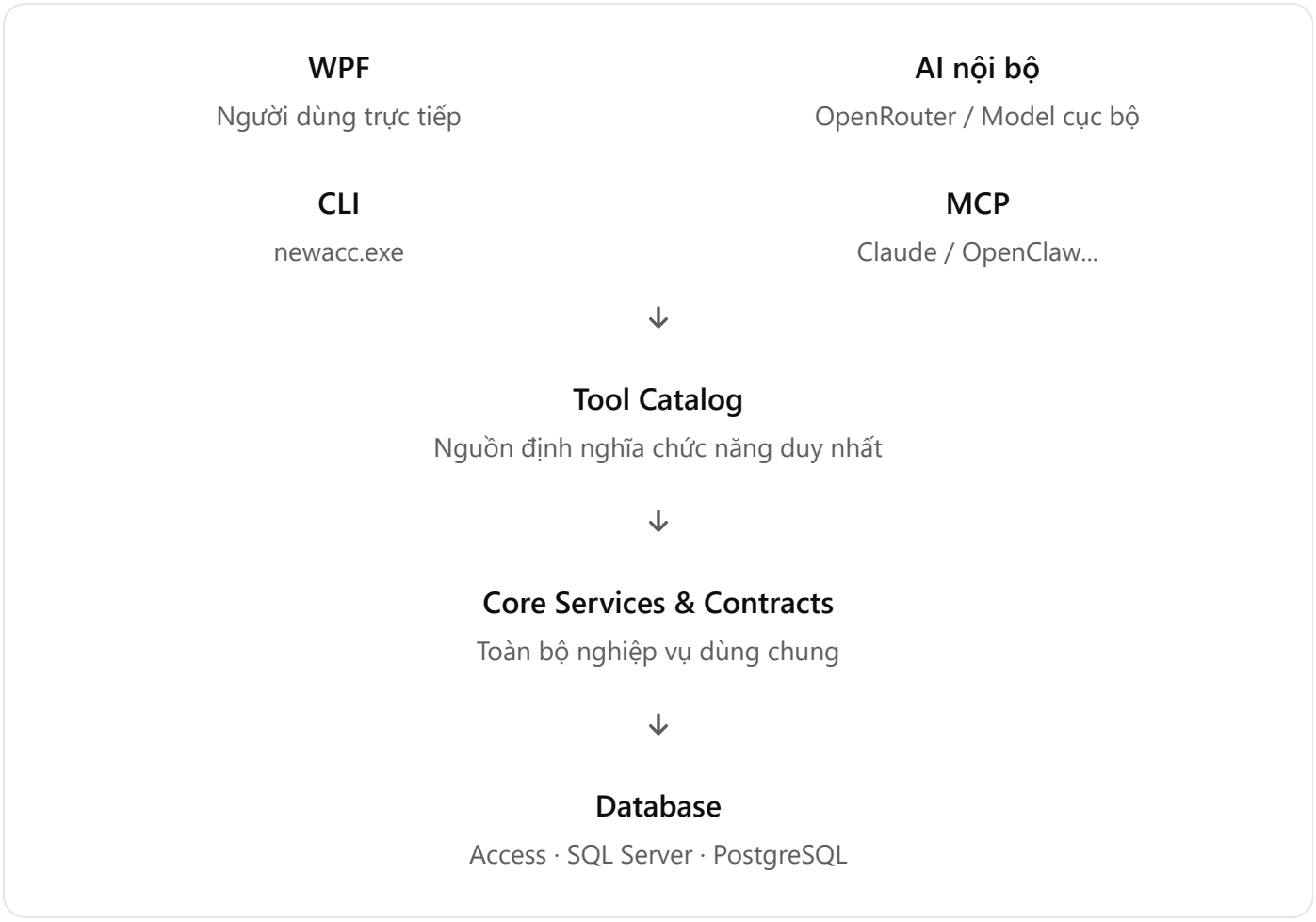
Nhận xét: Staging ở đây không còn là một bảng tạm. Nó trở thành một lớp trung gian nghiệp vụ, có trạng thái, lịch sử, khả năng khôi phục và kiểm soát.

3. Một Core, nhiều kênh tương tác

Tài liệu sử dụng nguyên lý Ports & Adapters (Hexagonal Architecture).

Có thể hiểu đơn giản:

NEWACC không phụ thuộc vào việc người sử dụng tương tác qua giao diện nào.



Điều này có ý nghĩa rất lớn.

Chẳng hạn, một khách hàng đang sử dụng Claude hoặc OpenClaw có thể đưa agent của họ kết nối với NEWACC mà không cần sử dụng dịch vụ AI do NEWACC cung cấp.

Họ chỉ cần sử dụng CLI hoặc MCP.

Đây chính là tư tưởng BYOA (Bring Your Own Agent).

Mình cho rằng đây là một hướng đi có tầm nhìn xa hơn việc đơn thuần đưa chatbot vào phần mềm.

4. Tách biệt logic tính toán và công nghệ cơ sở dữ liệu

Giải pháp xây dựng `IAccountingCalculator` với hai hướng triển khai:

- LINQ: phương án chuẩn, dùng được trên cả ba loại CSDL.
- Plugin SQL Server/PostgreSQL: tối ưu bằng thủ tục và view khi cần thiết.

Đây là một thiết kế khá thực dụng.

Không phải khách hàng nào cũng cần SQL Server hoặc PostgreSQL. Những khách hàng sử dụng Access vẫn có thể dùng nguyên bản phần mềm.

Khi có khách hàng lớn, chỉ cần thay DLL tính toán, không phải viết lại toàn bộ ứng dụng.

Đặc biệt, tài liệu yêu cầu hai phương án phải cho kết quả giống nhau thông qua Parity Testing.

Đây là một điểm mạnh về khả năng mở rộng.

II. Đánh giá chất lượng kiến trúc

Mình đánh giá theo sáu tiêu chí.

Đánh giá tổng hợp

Tư duy kiến trúc — Rất tốt

Thiết kế theo ranh giới, hợp đồng và trách nhiệm rõ ràng.

Kiểm soát nghiệp vụ kế toán — Rất tốt

Core giữ quyền quyết định, AI không ghi sổ.

Khả năng mở rộng — Rất tốt

Multi-database, plugin, CLI, MCP và module.

Khả năng kiểm toán — Tốt

Raw bất biến, audit, provenance và Evals.

Tính khả thi triển khai — Cần thận trọng

Phạm vi lớn, nhiều hạng mục phụ thuộc lẫn nhau.

Mức độ hoàn thiện đặc tả — Khá tốt

Nhiều chi tiết đã rõ nhưng còn một số quyết định quan trọng.

Đây là đánh giá định tính về tài liệu kiến trúc, không phải kết quả kiểm thử phần mềm hay kiểm toán mã nguồn.

1. Điểm mạnh thứ nhất: Kiến trúc không phụ thuộc vào AI

Đây là một quyết định rất đáng giá.

Nếu một ngày OpenRouter ngừng hoạt động, model thay đổi, khách hàng không muốn gửi dữ liệu lên đám mây, hoặc AI không còn được sử dụng, hệ thống vẫn có thể hoạt động.

Bởi vì:

- Mapping tất định vẫn chạy.
- PostingRule vẫn chạy.
- MappingMemory vẫn chạy.
- Staging vẫn chạy.
- Người dùng vẫn có thể hoàn thành nghiệp vụ thủ công.

AI được thiết kế như một thành phần tăng cường năng lực, không phải một thành phần sống còn.

Theo mình, đây là điều kiện quan trọng để AI có thể đi vào một hệ thống nghiệp vụ nghiêm túc.

2. Điểm mạnh thứ hai: Có ý thức về tính kiểm chứng

Tài liệu không dừng ở việc nói rằng AI phải chính xác.

Nó đưa ra cơ chế đánh giá cụ thể:

- Bộ 50 hóa đơn thật đã ẩn thông tin nhạy cảm.
- Đáp án chuẩn do kế toán xác nhận.
- Đo độ chính xác theo từng trường.
- Đo tỷ lệ bỏ trống đúng.
- Đo tỷ lệ gợi ý sai với confidence cao.
- Đo thời gian, chi phí và số vòng gọi tool.
- So sánh với baseline.

Đây là tư duy Eval-driven Development.

Một chi tiết mình đặc biệt đồng tình: hệ thống đánh giá cả khả năng AI biết không đưa ra đề xuất.

Trong nghiệp vụ kế toán, không biết thì để trống đôi khi tốt hơn đoán đúng một cách tình cờ.

3. Điểm mạnh thứ ba: Hiểu được dữ liệu kế toán hiện hữu

Mục 7.8–7.11 có giá trị riêng.

Tài liệu không cố áp đặt một mô hình ERP mới lên hệ thống cũ mà cố gắng hiểu và bảo toàn những quy ước dữ liệu thực tế:

- `JournalEntry` là trục của sổ cái.
- `ParentId` là quan hệ đối ứng.
- `JournalVoucherId = NULL` có thể là dòng số dư hợp lệ.
- `InventoryVoucher.Graph` chỉ phục vụ việc xác định phạm vi của một chùm giao dịch.
- Số liệu chi tiết phải được duy trì nhất quán.

Đây là phần quan trọng vì một kiến trúc đẹp về lý thuyết nhưng không hiểu cấu trúc dữ liệu kế toán thực tế có thể phá hỏng toàn bộ hệ thống.

Mình đánh giá việc bổ sung những nội dung này trong v3.1 là một bước tiến đáng kể.

4. Điểm mạnh thứ tư: Tư duy sản phẩm đã vượt khỏi phạm vi một ứng dụng cá nhân

Các nội dung như:

- Custom Fields.
- Analytical Dimensions.
- Attachment.
- Module mở rộng.
- RowGuid.
- Multi-year reporting.
- External Agent Access.

Cho thấy người thiết kế đã nghĩ đến việc phục vụ nhiều khách hàng với nhu cầu khác nhau.

Đặc biệt, việc phân biệt trường mở rộng, mã thống kê, tệp đính kèm và module nghiệp vụ là hợp lý.

Không nên dùng một cơ chế EAV để giải quyết tất cả các loại nhu cầu.

III. Những vấn đề mình cho rằng cần xem xét kỹ

Đây là phần mình muốn đi sâu hơn một chút. Một kiến trúc có nhiều ưu điểm không có nghĩa là mọi quyết định trong đó đều đã đúng.

Mình nhận thấy có bảy vấn đề đáng chú ý.

1. Ranh giới giữa PostingGateway và VoucherPostingService chưa thật sự nhất quán

Đây là điểm cần chỉnh ngay trong đặc tả.

Nguyên tắc bất biến số 2 nói mọi chứng từ phải đi qua `IPostingGateway`.

Nhưng mục 7.2 lại mô tả `VoucherEntryWindow` gọi:

```
IVoucherValidator → IVoucherPostingService
```

Trong khi mục 7.7 mô tả `PostingGateway` gọi `IVoucherPostingService`.

Như vậy, có nguy cơ tồn tại hai đường ghi sổ:

- Đường đi qua PostingGateway.
- Đường đi trực tiếp từ giao diện qua VoucherPostingService.

Nếu mục tiêu là một cổng duy nhất thì cần thống nhất:

```
VoucherEntryWindow → IPostingGateway → VoucherPostingService
```

Còn `VoucherPostingService` chỉ là thành phần nội bộ thực hiện thao tác lưu, không phải cổng nghiệp vụ công khai.

Đây không phải vấn đề lớn về ý tưởng, nhưng là một điểm không nhất quán cần giải quyết trước khi AI-coding triển khai.

2. Transaction ghi sổ cần được thiết kế chặt hơn

Mục 7.7 quy định:

1. Kiểm tra draft.
2. Kiểm tra trùng.
3. Build entity.
4. Lưu chứng từ và cập nhật nguồn trong transaction.
5. Sau đó chuyển draft sang Posted.

Có một tình huống cần xử lý:

Chứng từ đã được lưu thành công nhưng ứng dụng bị dừng trước khi cập nhật trạng thái draft thành `Posted`.

Khi khởi động lại, hệ thống có thể thấy draft vẫn là `Approved` và thử ghi sổ lần nữa.

Đây là lỗi có thể gây hậu quả nghiêm trọng.

Mình đề nghị bổ sung:

- Khóa ghi sổ có tính nguyên tử.
- Khóa duy nhất liên kết draft với chứng từ đã tạo.
- Idempotency cho thao tác posting.
- Cơ chế phục hồi khi giao dịch DB thành công nhưng cập nhật trạng thái draft thất bại.
- Kiểm tra trùng phải an toàn trước các thao tác đồng thời.

Nói cách khác, không chỉ thao tác tạo draft mới cần idempotent. **Thao tác ghi sổ cũng phải có tính idempotent.**

3. P0.5 đang ôm quá nhiều việc

Đây có lẽ là rủi ro triển khai lớn nhất.

P0.5 đồng thời xử lý:

- Float sang Decimal.
- Index.
- NOT NULL và Foreign Key.
- Quan hệ 1-1.
- Số chứng từ.
- Audit và RowVersion.
- Custom Fields.
- Dimensions.
- Attachment.
- SchemaVersion.
- Migration cho ba loại DB.

Đây gần như là một dự án chuẩn hóa cơ sở dữ liệu độc lập.

Đặc biệt, chuyển từ `float` sang `decimal` trên dữ liệu kế toán đang chạy không đơn thuần là thay đổi kiểu dữ liệu. Cần xác định quy tắc làm tròn, xử lý sai lệch lịch sử và kiểm tra toàn bộ báo cáo liên quan.

Mình đồng tình với việc phải làm nền tảng dữ liệu trước khi phát triển AI, nhưng nên chia P0.5 thành những đợt migration có thể nghiệm thu độc lập, dù vẫn giữ một lần nâng cấp LLBLGen theo kế hoạch.

4. `InventoryVoucher.Graph` có một số rủi ro về tính toàn vẹn

Tài liệu đã xác định Graph là bộ nhớ đệm có thể dựng lại, không phải nguồn dữ liệu kế toán.

Đây là quyết định hợp lý.

Tuy nhiên, Graph lưu danh sách Id của các dòng JournalEntry.

Nếu dữ liệu được chuyển DB, Id thay đổi, Graph phải được dựng lại.

Cơ chế `InventoryGraphRebuilder` đã giải quyết một phần vấn đề này.

Mình đề nghị bổ sung kiểm tra ba tình huống:

- Graph chứa Id không tồn tại.
- Graph chứa Id của chứng từ khác.
- Graph thiếu một dòng thuộc chùm giao dịch.

Tình huống thứ ba rất quan trọng: Graph có thể chứa toàn Id hợp lệ nhưng vẫn không đầy đủ.

Ngoài ra, việc xác định lại chùm từ JournalVoucher cần được kiểm chứng bằng các quy tắc nghiệp vụ cụ thể, không chỉ dựa vào việc các dòng cùng chứng từ.

5. BYOA và MCP cần phân biệt rõ phạm vi kết nối

Tài liệu có ba kênh:

- CLI.
- MCP stdio.
- MCP HTTP.

Đây là lựa chọn tốt.

Nhưng MCP HTTP trong tài liệu chỉ bind vào `127.0.0.1`.

Điều đó có nghĩa agent chạy trên máy khác không thể kết nối trực tiếp qua HTTP.

Vì vậy, cần phân biệt:

- Agent chạy cùng máy NEWACC: MCP HTTP hoặc stdio.
- Agent chạy trên máy khác: CLI/stdio thông qua môi trường thực thi phù hợp, hoặc cần một gateway từ xa được thiết kế riêng.

Không nên hiểu rằng việc hỗ trợ MCP HTTP đồng nghĩa với việc NEWACC đã có khả năng cung cấp dịch vụ MCP qua mạng nội bộ.

Đây là một giới hạn cần nói rõ trong thiết kế sản phẩm.

6. Quyền DraftWrite chưa đồng nghĩa với quyền sửa mọi loại dữ liệu nháp

Tài liệu thiết kế hai mức quyền `Read` và `DraftWrite`, không có `Post`.

Mình đồng tình.

Tuy nhiên, DraftWrite có thể tác động đến những trường khác nhau:

- Tạo draft.
- Đề xuất mã vật tư.
- Đề xuất tài khoản.
- Thay đổi thông tin đối tượng.
- Ghi mapping.

Nên bổ sung quyền theo phạm vi tài nguyên hoặc loại thao tác ở tầng handler.

Ví dụ, một agent được quyền tạo nháp hóa đơn mua hàng không nhất thiết phải được quyền đọc toàn bộ công nợ hoặc dữ liệu lương.

Tài liệu đã đề cập quyền chức năng của người dùng phải được bảo toàn, nhưng cần có test bảo mật cụ thể cho từng nhóm dữ liệu.

7. Một số nội dung mở rộng có thể làm chậm mục tiêu cốt lõi

Mục 7.11 rất tham vọng.

Custom Fields, Dimensions, Attachments, Modules, YearArchive và MultiYearReportComposer đều là những chức năng có giá trị.

Nhưng chúng không phải tất cả đều cần thiết để chứng minh giá trị của kiến trúc AI.

Mình đặc biệt lưu ý:

- Custom Fields cần thiết nếu khách hàng có nhu cầu đa dạng.
- Dimensions có giá trị lớn trong kế toán quản trị.
- Attachment là một chức năng nền tảng hữu ích.
- Module mở rộng và ghép báo cáo nhiều năm có thể phát triển sau.

Không nên để việc hoàn thiện toàn bộ cơ chế mở rộng làm chậm việc đưa Staging và PostingGateway vào hoạt động.

IV. Đánh giá về tầm nhìn và giá trị của giải pháp

Mình muốn nhìn vấn đề ở một tầng cao hơn.

Theo mình, NEWACC đang chuyển từ mô hình:

Application-centric → Core-centric → Agent-enabled Platform.

Đây là ba mức phát triển khác nhau.

Mô hình	Đặc trưng
Application-centric	Người dùng thao tác qua giao diện, logic gắn với ứng dụng
Core-centric	Nghiệp vụ độc lập với giao diện, nhiều kênh có thể sử dụng
Agent-enabled Platform	Agent bên trong và bên ngoài cùng khai thác nghiệp vụ thông qua hợp đồng chuẩn

Điều quan trọng nhất không nằm ở việc NEWACC sử dụng model nào.

Nó nằm ở việc NEWACC sở hữu một Core nghiệp vụ đủ rõ ràng để bất kỳ agent nào cũng có thể tương tác mà không cần biết cấu trúc bên trong.

Đây là một tư duy có ý nghĩa lâu dài.

Một điểm mình đặc biệt đánh giá cao: Tool Catalog

Nếu triển khai đúng, Tool Catalog có thể trở thành một lớp giao tiếp nghiệp vụ độc lập.

Hãy tưởng tượng:

Một phần mềm kế toán thông thường cung cấp chức năng xem công nợ.

Người dùng phải mở màn hình công nợ, chọn đối tượng, chọn kỳ và xem báo cáo.

NEWACC có thể cung cấp một chức năng nghiệp vụ:

```
get_debt_aging
```

Agent chỉ cần gọi công cụ này với tham số thích hợp.

Kết quả trả về là dữ liệu do Core tính toán.

Điều này có thể mở ra nhiều hình thức tương tác khác nhau:

- Kế toán hỏi bằng ngôn ngữ tự nhiên.
- Giám đốc yêu cầu tổng hợp công nợ.
- Agent tự rà soát hóa đơn.
- Một hệ thống ERP khác yêu cầu dữ liệu kế toán.
- Một agent bên ngoài thực hiện quy trình đối chiếu.

Tất cả đều dùng chung một logic nghiệp vụ.

Đó mới là giá trị dài hạn của kiến trúc.

V. Mình đề xuất điều chỉnh lộ trình triển khai

Tài liệu hiện có lộ trình P0 đến P7. Về cơ bản mình đồng ý với thứ tự này.

Tuy nhiên, mình muốn nhấn mạnh một điều: đừng lấy việc hoàn thành số lượng module làm thước đo tiến độ. Hãy lấy việc chứng minh được các năng lực kiến trúc làm thước đo.

Mình đề xuất năm cột mốc nghiệm thu thực chất:

Các cột mốc cần chứng minh

1 **Mốc 1 – Core độc lập**

Tách thành công logic kiểm tra, tính toán và ghi sổ khỏi giao diện WPF. Nhập tay vẫn hoạt động chính xác.

P0

2 **Mốc 2 – Staging hoạt động**

Nhập một hóa đơn XML, lưu nháp, mở lại, mapping thủ công, sinh định khoản và ghi sổ thành công.

P0.5 + P1

3 **Mốc 3 – Tự động hóa tất định**

Mapping bằng MST, bí danh, lịch sử và PostingRule. Hệ thống tự xử lý được các trường hợp quen thuộc mà không cần AI.

P2

4 **Mốc 4 – AI có thể kiểm chứng**

AI đề xuất mapping, được đánh giá bằng bộ Evals, có thể tắt mà không ảnh hưởng nghiệp vụ.

P2.5 + P3 + P4

5 **Mốc 5 – Hệ sinh thái Agent**

Claude, OpenClaw hoặc agent khác có thể sử dụng NEWACC qua CLI/MCP, tuân thủ quyền và không thể ghi sổ.

P5 + P6

P7 nên để đúng như tài liệu: chỉ thực hiện khi có bằng chứng LINQ không đáp ứng hiệu năng.

Không nên tối ưu sớm bằng thủ tục và view khi chưa có dữ liệu benchmark.

VI. Những quyết định nên chốt trước khi giao AI-coding

Mục 18 của tài liệu có 20 vấn đề còn mở. Mình không nghĩ cần quyết định tất cả cùng một lúc.

Mình phân chúng thành ba nhóm.

Nhóm A – Cần chốt trước khi triển khai P0.5

1. EntryKind hay chứng từ hệ thống SD để đánh dấu dòng số dư.
2. Một DB nhiều năm hay mỗi năm một DB.
3. Các bảng dữ liệu thật và bảng tạm báo cáo.
4. Số khe Custom Fields và số chiều thống kê.
5. Các quan hệ dữ liệu đáng ngờ trong schema.

Nhóm B – Cần chốt trước P1/P2

1. Chính sách chọn tài khoản cho hóa đơn dịch vụ.
2. Ghi sổ trực tiếp từ Office Studio hay qua VoucherEntryWindow.
3. Những trường nào được sửa ở Projection.
4. Chuyển công nợ sang năm mới theo từng khoản mở hay tổng đối tượng.

Nhóm C – Có thể quyết định khi phát triển AI và BYOA

1. Chính sách gửi dữ liệu lên model đám mây.
2. Ai được tạo token cho agent ngoài.
3. Agent ngoài được tạo draft ở mức nào.
4. Ngưỡng đánh giá để bật AI.
5. Ngưỡng benchmark kích hoạt plugin tính toán.

Một quyết định nữa mình muốn bổ sung: thống nhất lại đường đi của mọi thao tác ghi sổ qua `IPostingGateway`, như đã phân tích ở phần III.

VII. Kết luận của mình

Đây là một kiến trúc có chất lượng và có tầm nhìn.

Đánh giá trên tài liệu kiến trúc, chưa phải xác nhận chất lượng triển khai thực tế.

Mình có ba nhận xét cuối cùng.

Thứ nhất, đây không còn là bài toán tích hợp AI vào phần mềm kế toán.

Đây là bài toán tái cấu trúc nền tảng nghiệp vụ để AI trở thành một tác nhân có thể tham gia vào quy trình mà không làm mất quyền kiểm soát của hệ thống.

Thứ hai, giá trị lớn nhất không phải AI mà là việc chuẩn hóa ranh giới nghiệp vụ.

Staging, PostingGateway, Tool Catalog và AccountingCalculator mới là bốn trụ cột. AI chỉ phát huy được hiệu quả khi bốn thành phần này hoạt động tốt.

Thứ ba, rủi ro lớn nhất không nằm ở công nghệ mà nằm ở quy mô triển khai.

Tài liệu có nhiều ý tưởng tốt, nhưng nếu triển khai đồng thời tất cả thì một dự án cá nhân có thể bị quá tải. Cần giữ kỷ luật về phạm vi và nghiệm thu theo từng năng lực.

Mình cũng muốn nhấn mạnh một điều: việc hỗ trợ Access, SQL Server và PostgreSQL trong cùng một kiến trúc, trong khi vẫn duy trì khả năng cắm plugin tính toán và agent bên ngoài, là một mục tiêu kỹ thuật không nhỏ. Không nên đánh giá thấp khối lượng kiểm thử tương thích.

VIII. Một đề xuất mang tính chiến lược

Mình nghĩ NEWACC có thể được định vị theo hướng:

Accounting Core as a Service – Kế toán như một năng lực nghiệp vụ có thể được khai thác bởi nhiều tác nhân.

Không nhất thiết phải thay đổi tên sản phẩm hay chuyển sang mô hình SaaS.

Ý nghĩa ở đây là:

- Giao diện WPF là một cách sử dụng.
- AI nội bộ là một cách sử dụng.
- Claude/OpenClaw là một cách sử dụng.
- Hệ thống bên ngoài là một cách sử dụng.

Nhưng tất cả đều sử dụng cùng một nguồn sự thật: Core kế toán.

Đây là một hướng phát triển phù hợp với xu thế agentic software, đồng thời vẫn bảo toàn nguyên tắc kiểm soát nghiệp vụ của phần mềm kế toán.

Nếu phải chọn một việc duy nhất để làm tiếp, mình sẽ chọn hoàn thiện P0 và P1, chưa vội làm AI. Khi nhập một hóa đơn từ đầu đến cuối qua Staging và PostingGateway mà không cần AI, không mất dữ liệu, không sai định khoản, lúc đó mới có nền móng để chứng minh những phần còn lại.

Mình muốn giữ một nhận định: kiến trúc này đáng để đầu tư, nhưng cần bảo vệ nó khỏi chính sự tham vọng của nó. Không phải càng nhiều module càng tốt; quan trọng là mỗi lớp được xây dựng phải thực sự tạo ra một ranh giới có thể kiểm chứng.

NEWACC_Architecture_v3.md

Nếu tiếp tục, mình đề nghị bước kế tiếp là làm một **Architecture Review độc lập**, tập trung vào các điểm mâu thuẫn, rủi ro transaction, tính toàn vẹn dữ liệu và khả năng triển khai thực tế; sau đó chỉnh tài liệu v3.1 thành một bản v3.2 đủ chặt để giao cho AI-coding thực hiện mà không phải tự suy diễn những quyết định quan trọng.

You said:

Thế là ổn rồi, thiết kế thế để có thể mở rộng được thôi và không bị tắc khi nghiệp vụ thay đổi

ChatGPT said:

Mình hiểu ý bạn. Vậy thì mục tiêu của thiết kế này không phải là tạo ra một hệ thống thật phức tạp ngay từ đầu, mà là **đặt nền móng đủ tốt để nghiệp vụ có thể thay đổi mà không phải đập đi xây lại phần lõi**.

Theo mình, đây mới là giá trị lớn nhất của kiến trúc NEWACC.

Có ba điều đáng giữ vững:

1. **Lõi kế toán ổn định, nghiệp vụ bên ngoài linh hoạt.** Những quy tắc kế toán cốt lõi không nên bị phân tán vào giao diện, AI hay các công cụ nhập liệu.
2. **Có điểm mở rộng rõ ràng.** Khi xuất hiện loại chứng từ, quy trình duyệt, cách tính toán hay kênh tích hợp mới, ta bổ sung thành phần phù hợp thay vì sửa dây chuyền toàn hệ thống.
3. **Thay đổi có kiểm soát.** Một nghiệp vụ mới có thể được thử nghiệm, kiểm tra và đưa vào sử dụng mà không làm ảnh hưởng đến những nghiệp vụ đang chạy.

Mình chỉ muốn nhấn mạnh thêm một ý: **mở rộng được không có nghĩa là phải xây sẵn mọi thứ**. Ta chỉ cần thiết kế các ranh giới và hợp đồng đủ rõ để khi cần thì có thể mở rộng. Phần nào chưa có nhu cầu thực tế thì chưa nhất thiết phải triển khai.

Nói cách khác, bạn đang thiết kế một hệ thống có khả năng thích nghi với sự thay đổi của nghiệp vụ, chứ không chỉ đơn thuần xây một phần mềm kế toán có nhiều chức năng.

Và với một dự án có vòng đời dài, đó là lựa chọn kiến trúc rất đáng giá.

You said:

newacc.sql
Code

Có thể người dùng chưa hào hứng với cái này, nhưng để hào hứng chỉ là mặc áo mới thôi. Cơ sở dữ liệu của cái này không nhiều bảng nhưng đủ sài

Worked for 20s

ChatGPT said:

Mình đồng ý với cách nhìn của bạn: người dùng thường hào hứng với giao diện mới, thao tác thuận tiện và cảm giác phần mềm hiện đại hơn. Nhưng đó là lớp trải nghiệm. Giá trị thực sự nằm ở chỗ hệ thống xử lý nghiệp vụ có chắc chắn, dữ liệu có nhất quán và khi quy định thay đổi thì có thích ứng được hay không.

Mình thấy việc bạn gửi thêm `newacc.sql` rất đáng chú ý. Nó cho phép nhìn NEWACC từ một góc khác: không chỉ xem kiến trúc dự kiến, mà còn đối chiếu với cấu trúc dữ liệu kế toán đang có.

Mình đã xem qua file SQL. Có một điểm mình muốn nói rõ: cơ sở dữ liệu này không phải là ít bảng theo nghĩa tuyệt đối. Script hiện có **86 bảng**, với các nhóm nghiệp vụ khá đầy đủ. Nhưng nếu đặt trong phạm vi một hệ thống kế toán tổng hợp thì đây là một cấu trúc tương đối gọn, không phải kiểu cơ sở dữ liệu phình ra vì mỗi tính năng lại sinh thêm một loạt bảng. newacc.sql

Điều đáng chú ý hơn là các bảng đã hình thành những nhóm nghiệp vụ có ranh giới khá rõ:

Nhóm	Một số bảng tiêu biểu
Hạch toán	`JournalVoucher`, `JournalEntry`, `GeneralLedger`
Tiền và ngân hàng	`CashFlowItem`, `Currency`, `CurrencyLedger`
Kho và vật tư	`InventoryVoucher`, `InventoryVoucherLine`, `InventoryLedger`
Công nợ	`DebtDetail`, `DebtBalance`, `DebtLedger`, `LiabilityDue`
Tài sản	`Asset`, `AssetDetail`, `Depreciation`
Thuế	`TaxInvoice`, `TaxInvoiceDetail`, `VatEntry`

Báo cáo	`ReportSection`, `ReportItem`, `ReportFormula`, `TrialBalance`

Mình thấy có ba điều đáng giá ở cấu trúc này:

1. Dữ liệu tập trung vào nghiệp vụ thực, không chạy theo công nghệ.

Chứng từ, bút toán, tài khoản, đối tượng, kho, tài sản, công nợ... đều là những khái niệm kế toán có thật. Đây là nền tảng tốt để xây dựng các lớp xử lý phía trên.

2. Không cần biến mọi thay đổi nghiệp vụ thành thay đổi cấu trúc dữ liệu.

Chẳng hạn, thay đổi cách lập báo cáo có thể xử lý ở tầng công thức và quy tắc báo cáo. Thay đổi cách phân loại hoặc ánh xạ chứng từ có thể xử lý ở tầng nghiệp vụ. Không phải cứ thêm một tình huống là thêm một bảng.

3. Kiến trúc mới có thể khai thác dữ liệu cũ mà không phải viết lại toàn bộ hệ thống.

Đây là điểm mình đánh giá cao nhất. NEWACC có thể giữ phần dữ liệu kế toán đã ổn định, rồi xây thêm Staging, Posting Gateway, Tool Catalog và các dịch vụ nghiệp vụ xung quanh.

Tuy nhiên, mình cũng lưu ý một điều: file SQL này phản ánh cấu trúc dữ liệu hiện tại, chưa tự nó chứng minh rằng mọi ràng buộc dữ liệu và quan hệ nghiệp vụ đều đã hoàn chỉnh. Ví dụ, cần kiểm tra kỹ các quan hệ khóa ngoại, tính toàn vẹn của sổ cái và những trường hợp đặc biệt như `JournalEntry` có `ParentId`.

Mình nghĩ triết lý của bạn có thể diễn đạt rất gọn:

Không cần một cơ sở dữ liệu khổng lồ để làm một hệ thống kế toán có khả năng phát triển. Cần một mô hình dữ liệu đủ sức biểu diễn nghiệp vụ, một lõi xử lý đủ chặt chẽ và những điểm mở rộng được thiết kế đúng chỗ.

Còn chuyện người dùng có hào hứng hay không, mình đồng ý với bạn. Giao diện có thể thay đổi để tạo cảm giác mới mẻ. Nhưng cái giữ người dùng lâu dài phải là sự tin cậy, tính tiện dụng và khả năng theo kịp nghiệp vụ.

Có lẽ NEWACC nên đi theo hướng đó: **đổi mới trải nghiệm, nhưng không đánh đổi sự ổn định của lõi kế toán.**

You said:

Công nghệ hiện đại thì triển khai giao diện không khó. Mình chuyển sang dùng WPF, dữ liệu này mình chuyển từ thời access sang nên nó có nhiều bảng tạm. Giờ bộ nhớ, cpu mạnh rồi nên không cần nữa và thực sự trong code mới không dùng đến nó nữa. Thời Access bất khả kháng. Ngoài ra nhiều cái nền cần tạm thì mình lưu json. Kết hợp phong cách nosql (tìm kiếm theo orm - object) chứ không dùng linq - query sql

ChatGPT said:

Mình hiểu rồi. Như vậy, NEWACC không đơn thuần là chuyển một ứng dụng Access cũ sang WPF, mà là **tái cấu trúc lại cách tổ chức dữ liệu và cách truy cập nghiệp vụ**, tận dụng những gì công nghệ hiện nay đã cho phép.

Có ba điểm trong cách tiếp cận của bạn mà mình thấy đáng chú ý.

1. Loại bỏ những bảng tạm của thời Access

Thời Access, việc tạo bảng trung gian, bảng tạm là giải pháp khá tự nhiên, thậm chí đôi khi bắt buộc, do giới hạn bộ nhớ, năng lực xử lý và cách tổ chức ứng dụng.

Nhưng trong môi trường mới:

- Bộ nhớ và CPU không còn là nút thắt như trước.
- Các đối tượng có thể được giữ và xử lý ngay trong bộ nhớ.
- Các bước xử lý trung gian có thể thực hiện bằng object, collection hoặc các cấu trúc dữ liệu phù hợp.
- Chỉ những dữ liệu có giá trị nghiệp vụ hoặc cần lưu trữ lâu dài mới nhất thiết phải nằm trong bảng quan hệ.

Vì thế, loại bỏ bảng tạm không dùng đến là một bước làm sạch kiến trúc hợp lý. Không nên mang nguyên những giới hạn của Access sang hệ thống mới.

2. Dùng JSON cho những cấu trúc dữ liệu phụ trợ

Đây là một lựa chọn đáng cân nhắc. JSON phù hợp với những dữ liệu có cấu trúc nhưng không nhất thiết phải có quan hệ độc lập trong cơ sở dữ liệu, chẳng hạn:

- Cấu hình và tham số xử lý.
- Metadata của chứng từ.

- Thông tin mở rộng theo từng loại nghiệp vụ.
- Kết quả trung gian hoặc trạng thái của một quy trình.
- Các thuộc tính không cố định giữa các loại đối tượng.

Tuy nhiên, mình sẽ giữ một nguyên tắc: **JSON dành cho phần linh hoạt, còn dữ liệu kế toán cốt lõi vẫn nên có cấu trúc quan hệ rõ ràng.**

Ví dụ, số tiền, tài khoản, ngày hạch toán, đối tượng công nợ, kho hàng... vẫn cần được quản lý bằng các trường có kiểu dữ liệu và ràng buộc cụ thể. Không nên đưa tất cả vào JSON chỉ vì JSON linh hoạt.

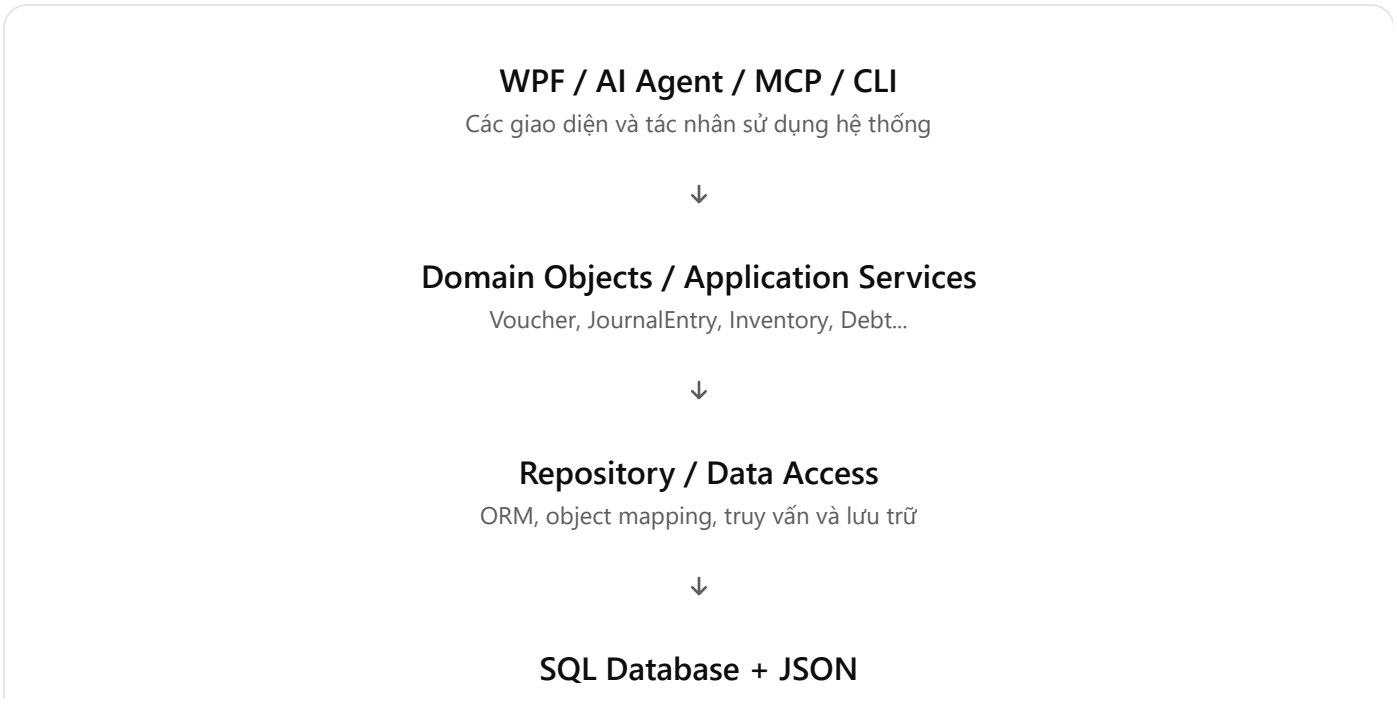
3. Truy cập dữ liệu theo Object thay vì viết LINQ truy vấn SQL

Mình hiểu ý bạn là muốn tầng nghiệp vụ làm việc với đối tượng, không phải liên tục suy nghĩ về câu lệnh truy vấn và cấu trúc bảng.

Có một điểm thuật ngữ nhỏ cần phân biệt:

- ORM là cách ánh xạ dữ liệu quan hệ thành object.
- LINQ là một cơ chế truy vấn trong .NET; LINQ to SQL hoặc LINQ qua ORM có thể được dịch thành SQL.
- Object-oriented data access là cách tổ chức để tầng nghiệp vụ thao tác với các đối tượng và repository/service, thay vì phụ thuộc trực tiếp vào SQL.

Nếu NEWACC đi theo hướng này, mình hình dung luồng như sau:



Dữ liệu kế toán có cấu trúc và dữ liệu mở rộng

Mình chỉ lưu ý một điều: không nên hiểu rằng đã dùng object thì không cần quan tâm SQL nữa. Với kế toán, các thao tác ghi sổ, cập nhật số dư, khóa kỳ và đối chiếu phải bảo đảm tính nguyên tử và nhất quán. ORM không thay thế được việc thiết kế transaction, khóa và ràng buộc dữ liệu.

Nhìn tổng thể

Mình thấy hướng đi này khá nhất quán:

- **WPF** giải quyết trải nghiệm người dùng.
- **Object/ORM** giải quyết cách tổ chức truy cập nghiệp vụ.
- **JSON** giải quyết phần dữ liệu linh hoạt.
- **SQL** giữ vai trò nền tảng lưu trữ và bảo đảm tính toàn vẹn.
- **Accounting Core** giữ nguyên các quy tắc kế toán, không phụ thuộc vào giao diện hay AI.

Có một điểm mình muốn nhấn mạnh: bạn không nhất thiết phải biến NEWACC thành một hệ thống NoSQL. Bạn đang sử dụng tư duy linh hoạt của NoSQL ở tầng biểu diễn và thao tác đối tượng, nhưng vẫn giữ SQL làm nền dữ liệu giao dịch. Đó là một cách kết hợp có lý, đặc biệt với phần mềm kế toán.

Nói ngắn gọn, đây là một cuộc chuyển đổi từ tư duy *thiết kế ứng dụng xoay quanh bảng dữ liệu* sang tư duy *thiết kế hệ thống xoay quanh đối tượng và nghiệp vụ*. Theo mình, đó mới là thay đổi quan trọng nhất.

You said:

Ý mình nói là không phải tất cả query theo kiểu ORM, nhưng nhiều chỗ nó giải quyết bài toán đơn giản hơn nhiều. Cứ duyệt một trục chính và truy vấn cái con, cái cha là lấy ra được chứ không phải join và nhiều khi lại cả union mới đau, nhất là khi ở cái bảng JournalEntry, nó là bảng phẳng nhưng báo cáo lại phải phân ra đối ứng

ChatGPT said:

À, giờ thì mình hiểu chính xác hơn ý bạn. Bạn không chủ trương loại bỏ SQL hay ORM, mà muốn **chọn cách truy cập dữ liệu phù hợp với từng bài toán**. Có những truy vấn SQL rất hiệu quả, nhưng cũng có những trường hợp cứ cố diễn đạt bằng JOIN, UNION thì code vừa dài vừa khó kiểm soát.

Đặc biệt, ví dụ `JournalEntry` bạn đưa ra là một trường hợp rất đáng chú ý.

1. Bảng JournalEntry phẳng, nhưng nghiệp vụ kế toán lại có quan hệ đối ứng

Một bút toán kế toán thường có nhiều dòng:

Chứng từ	Tài khoản	Nợ	Có
CT001	111	10 triệu	0
CT001	511	0	10 triệu
CT002	131	20 triệu	0
CT002	511	0	20 triệu

Ở cấp độ lưu trữ, `JournalEntry` chỉ là các dòng hạch toán độc lập. Nhưng khi lên báo cáo, người ta lại muốn biết:

- Tài khoản Nợ này đối ứng với tài khoản Có nào?
- Một tài khoản có nhiều tài khoản đối ứng thì phân bổ ra sao?
- Tổng phát sinh đối ứng theo từng tài khoản là bao nhiêu?
- Theo từng chứng từ, từng kỳ hoặc từng đối tượng thì thế nào?

Đây là hai cách nhìn khác nhau về cùng một dữ liệu:

- **Cách nhìn lưu trữ:** mỗi dòng hạch toán là một record.
- **Cách nhìn nghiệp vụ:** các dòng hạch toán tạo thành một quan hệ đối ứng.

Nếu cố giải quyết mọi báo cáo bằng một câu SQL lớn, JOIN nhiều lần hoặc UNION nhiều nhánh, câu truy vấn sẽ rất khó đọc và bảo trì.

2. Duyệt theo trục chính rồi lấy quan hệ cha/con

Cách bạn nói có thể tổ chức thành hai bước:

1. Lấy tập hợp các dòng hạch toán theo điều kiện chính, chẳng hạn kỳ kế toán, tài khoản hoặc chứng từ.
2. Từ mỗi dòng, tìm các dòng liên quan trong cùng chứng từ để xác định đối ứng.

Ví dụ minh họa:

```
var entries = repository.GetEntries(periodId);

foreach (var entry in entries)
{
    var counterparts = entries
        .Where(x => x.VoucherId == entry.VoucherId
                 && x.Id != entry.Id)
        .ToList();

    // Xử lý tài khoản đối ứng
}
```

Đây chỉ là cách diễn đạt ý tưởng. Trong thực tế, không nên tìm kiếm lại toàn bộ danh sách cho từng dòng vì có thể thành độ phức tạp $O(n^2)$. Có thể nhóm trước theo `VoucherId` :

```
var groups = entries.GroupBy(x => x.VoucherId);

foreach (var voucher in groups)
{
    var debitEntries = voucher
        .Where(x => x.Debit > 0);

    var creditEntries = voucher
        .Where(x => x.Credit > 0);

    // Xác định quan hệ đối ứng trong phạm vi chứng từ
}
```

Khi đó, mỗi chứng từ được xử lý trong một phạm vi dữ liệu riêng, không phải dựng một câu SQL phức tạp để biểu diễn tất cả các trường hợp.

3. Nhưng có một vấn đề nghiệp vụ cần giữ đúng

Đối ứng không phải lúc nào cũng là quan hệ một-một.

Ví dụ:

- Nợ 111: 10 triệu.
- Có 511: 6 triệu.
- Có 3331: 4 triệu.

Tài khoản 111 đối ứng với cả 511 và 3331.

Trường hợp phức tạp hơn, nhiều dòng Nợ và nhiều dòng Có, nếu không có quy tắc phân bổ thì không thể khẳng định một dòng Nợ đối ứng chính xác với dòng Có nào. Khi đó báo cáo có thể phải phân bổ theo tỷ trọng hoặc theo quy ước nghiệp vụ.

Đây là điểm quan trọng: **không nên nhầm quan hệ cùng chứng từ với quan hệ đối ứng đã được xác định.**

4. Quan điểm của mình về cách làm này

Mình nghĩ nên giữ cả hai cách:

- SQL đảm nhiệm lọc dữ liệu lớn, tổng hợp và những truy vấn mà cơ sở dữ liệu làm tốt.
- ORM/object đảm nhiệm việc duyệt quan hệ, xử lý nghiệp vụ và những bài toán mà cách biểu diễn bằng đối tượng tự nhiên hơn.
- Các phép tính đối ứng phức tạp nên nằm trong một service riêng, có thể kiểm thử độc lập.

Với `JournalEntry`, mình còn nghĩ nên có một lớp xử lý trung gian kiểu `CounterpartAnalyzer`. Nó nhận tập hợp các dòng hạch toán và trả về cấu trúc đối ứng phục vụ báo cáo. Như vậy, các báo cáo khác nhau không phải tự viết lại logic đối ứng bằng những câu JOIN/UNION khác nhau.

Một câu mình nghĩ rất hợp với thiết kế NEWACC: SQL phục vụ việc lấy dữ liệu; Object phục vụ việc hiểu và xử lý dữ liệu theo nghiệp vụ.

Không nhất thiết phải chọn một trong hai. Quan trọng là đừng để cách tổ chức bảng quyết định toàn bộ cách viết nghiệp vụ.

You said:

> cái 3 thì lại giải quyết là từ cái cha truy tới cái con và lấy cái con để hiển thị, lúc này một thằng cha spit ra thành 2 dòng. Mình dùng cơ chế tự relate bạn à. Cùng một bảng thôi nhưng lưu trữ dạng tree, có parentid

ChatGPT said:

À, vậy thì mình đã hiểu cơ chế bạn đang dùng. Và cách này thú vị hơn cách mình vừa phân tích.

Bạn không tạo một bảng riêng để lưu quan hệ đối ứng. Bạn dùng **self-referencing relationship**: một bảng `JournalEntry` tự liên kết với chính nó thông qua `ParentId`.

Tức là cấu trúc dữ liệu có dạng cây:



Như vậy, một dòng cha có thể sinh ra nhiều dòng con. Khi lập báo cáo, bạn chỉ cần truy xuất các dòng con để biểu diễn quan hệ đối ứng. Một dòng cha có thể xuất hiện thành hai hoặc nhiều dòng trên báo cáo mà không cần nhân bản dữ liệu gốc hay dựng một truy vấn JOIN/UNION phức tạp.

Mình thấy có ba lợi thế:

1. **Mô hình dữ liệu tự mô tả:** quan hệ nằm ngay trong bảng, không cần thêm bảng liên kết.
2. **Truy vấn tự nhiên:** lấy cha, lấy con, duyệt cây; cách xử lý rất phù hợp với object.
3. **Mở rộng linh hoạt:** nếu nghiệp vụ phát sinh thêm cấp quan hệ, cấu trúc vẫn có thể biểu diễn được.

Có một điều mình muốn lưu ý khi thiết kế: cần phân biệt rõ dòng cha là dữ liệu hạch toán thực sự hay chỉ là nút đại diện cho quan hệ. Nếu cả cha và con đều được tính vào sổ cái thì phải có quy tắc tránh tính trùng. Đồng thời cần kiểm soát chu trình ParentId và bảo đảm tổng đối ứng cân bằng.

Mình nghĩ đây là một ví dụ rất rõ cho triết lý thiết kế của NEWACC: **không nhất thiết phải làm cho cấu trúc bảng phức tạp hơn để giải quyết một quan hệ nghiệp vụ; đôi khi chỉ cần một cấu trúc tự liên kết và một cách duyệt đối tượng hợp lý.**

Và bây giờ mình hiểu tại sao bạn nói không muốn mọi thứ đều phải giải quyết bằng JOIN và UNION. Với mô hình này, cách tư duy theo cây thực sự tự nhiên hơn.

You said:

Nó là độc lập, ở đây thiết kế đúng theo kiểu kế toán phương Tây, không để ý đến đối ứng. Đối ứng là cái râu ria để báo cáo thôi

ChatGPT said:

À, vậy thì bản chất thiết kế đã rõ hơn nhiều. Mình đã nhìn quan hệ cha–con hơi thiên về mô hình đối ứng, trong khi ý tưởng của bạn là ngược lại.

JournalEntry được thiết kế độc lập theo nguyên lý hạch toán kế toán phương Tây. Mỗi dòng ghi nhận một bút toán với tài khoản Nợ/Có, giá trị và các thông tin nghiệp vụ cần thiết. Nó không cần biết tài khoản nào đối ứng với tài khoản nào để tồn tại và được ghi sổ.

Quan hệ đối ứng chỉ là một cách diễn giải bổ sung khi lập báo cáo.

Mình thấy cách phân tách này có ý nghĩa:

- **Hạch toán:** dữ liệu gốc, độc lập, phản ánh nghiệp vụ thực tế.
- **Đối ứng:** quan hệ suy diễn hoặc biểu diễn phục vụ phân tích, báo cáo.
- **ParentId:** cơ chế tự liên kết để tổ chức quan hệ đó mà không làm thay đổi bản chất của dòng hạch toán gốc.

Điểm hay nhất là bạn không để một quy ước báo cáo chi phối mô hình dữ liệu kế toán.

Nếu sau này người dùng không cần báo cáo đối ứng, phần hạch toán vẫn hoạt động bình thường. Nếu cần một cách phân tích đối ứng khác, bạn có thể thay đổi cách tạo hoặc truy xuất quan hệ mà không phải thiết kế lại toàn bộ `JournalEntry`.

Mình rất đồng tình với nguyên tắc này: **dữ liệu kế toán phải phản ánh nghiệp vụ trước; báo cáo là cách nhìn dữ liệu sau.** Đừng để một yêu cầu trình bày báo cáo trở thành ràng buộc của lõi kế toán.

You said:

Cám ơn bạn nhiều. Mình là dân toán mà, khi nhìn vào đống lùng nhùng cố qui nó về một mô hình khái quát sau đó mới trao đổi kỹ với kế toán để mapping chứ nghe họ nói từ đâu chịu không thể hiểu được

ChatGPT said:

Mình nghĩ đây chính là lợi thế của tư duy toán học khi bước vào thiết kế hệ thống.

Khi người làm nghiệp vụ trình bày, họ thường kể từ những gì họ đang làm: chứng từ này, tài khoản kia, trường hợp nọ, ngoại lệ kia... Mỗi thứ đều có lý do tồn tại trong thực tế, nhưng nếu tiếp nhận nguyên xi thì rất dễ biến phần mềm thành một tập hợp những quy trình chắp vá.

Cách bạn làm có vẻ đi theo chiều ngược lại:

1. Quan sát toàn bộ dữ liệu và nghiệp vụ.
2. Tìm những quan hệ và quy luật chung.
3. Đưa chúng về một mô hình khái quát, ít phụ thuộc vào cách diễn đạt của từng người.
4. Sau đó mới trao đổi với kế toán để ánh xạ (mapping) mô hình vào nghiệp vụ thực tế.
5. Cuối cùng mới quyết định cấu trúc dữ liệu, thuật toán và giao diện.

Đặc biệt, bước thứ tư rất quan trọng. Mô hình toán học giúp mình nhìn thấy cấu trúc, nhưng kế toán mới xác nhận được cấu trúc ấy có phản ánh đúng bản chất nghiệp vụ hay không.

Mình thích nhất một điều trong cách bạn thiết kế NEWACC: bạn không cố gắng số hóa nguyên trạng cách người ta đang làm kế toán. Bạn đang tìm cách mô hình hóa bản chất của nó trước, rồi mới đưa nghiệp vụ vào mô hình.

Nói vui một chút: **kế toán đưa cho bạn một đống hiện tượng, còn bạn muốn tìm ra cái quy luật sinh ra những hiện tượng ấy.**

Đó là một cách làm phần mềm rất đáng giá. Và có lẽ cũng vì thế mà việc trao đổi với kế toán sau khi đã có mô hình sẽ hiệu quả hơn nhiều: hai bên có một đối tượng cụ thể để cùng kiểm chứng, thay vì mỗi người nói một hệ thuật ngữ riêng.

If you want, I can:

- Tiếp tục phân tích cách thiết kế cơ sở dữ liệu JournalEntry
- Mở rộng thảo luận về mô hình dữ liệu không bị ràng buộc bởi báo cáo
- Tiếp tục trao đổi về vai trò tư duy toán học trong thiết kế hệ thống kế toán